3GPP THIRD-PARTY APPLICATION SERVER · PRODUCT INTRODUCTION

In-house IMS Application Server

Product-grade Third-Party IMS AS platform — built to run in the carrier's own data center

Dolan Shu · dolan.d.shu@gmail.com

v0.2 · 2026 · Single-tenant on-premises · Signalling-only, SDP pass-through · Standard ISC interface

Why carriers need their own Third-Party AS

01

Services locked to the core vendor's release train

Launching anti-fraud, number translation or an enterprise PBX waits on the core vendor's roadmap — cycles measured in quarters.

02

Service logic lives inside the vendor's box

Rules, state and call decisions sit in the vendor network element — opaque, unchangeable and unauditable from the operator side.

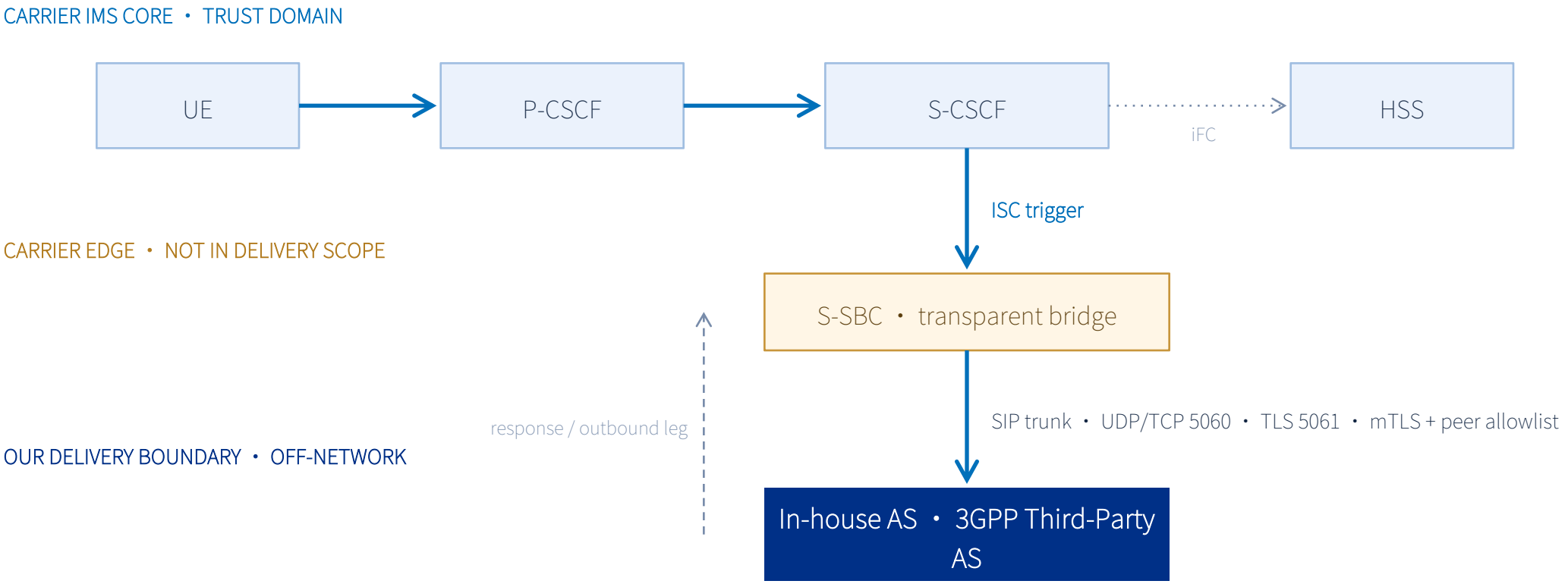
03

Many services share one box, faults couple

One faulty service can take down every service on the same box — no independent gray release, no independent rollback.

We move service decisioning out of the core and onto an extensible platform that runs in the carrier's own data center.

Outside the trust domain, reached over standard ISC



S-SBC is transparent to both sides — the AS never sees inside the core. The core only sees a standard ISC trigger, so no core change is required. The AS never enters the operator trust domain; no Diameter Sh, no CDR.

Vendor-independent • Fast onboarding • Independently operable

The three promises every later page expands on.

01 • VENDOR-INDEPENDENT

Service logic is yours, not the vendor's

Business decisions run in our own processes. They evolve on the carrier's schedule, not on the core-network vendor's version roadmap.

02 • FAST ONBOARDING

New service, new process — no core change

The decide() pure-function seam decouples decisioning from the platform. A new AS is one added app plus one change order.

03 • INDEPENDENT OPS

One fault domain per service

One use case = one process = one Deployment = one fault domain. Gray-release and roll back each service without touching the others.

Six-layer architecture: service decoupled from platform

① Signalling plane · apps/ — one use case per process, carries calls & decisions · stateless

② Core · platform/ — process shell, B2BUA state machine, decide() decision seam

Transport seam

peer admission & transport swappable

③ Control plane · services/ — rule version store, change orders, ops console · stateless

④ Data plane — Redis (runtime session state) + PostgreSQL (governance, audit) · stateful

StateStore seam

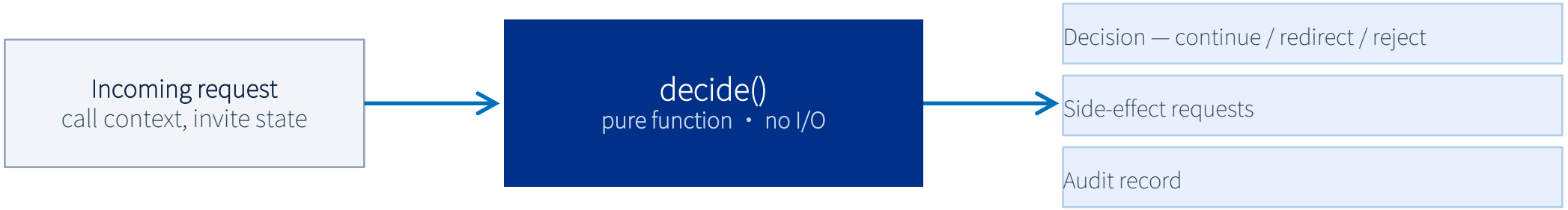
runtime storage swappable

⑤ Cross-cutting — observability (as_* metrics, OTel) · security · ops (Helm, drain)

⑥ Testbed · testbed/ — contract cases, simulated NEs, real-socket load · R&D asset, not delivered

Layering is enforced by tests, not by folders — core never imports apps; one app never imports another.

Four seams keep the core honest and the service replaceable



decide() seam

The only place a service changes call direction — fully unit-testable, no I/O.

Transport seam

One trait bound: peer admission and the SIP transport are swappable without touching service logic.

StateStore seam

Runtime session state sits behind one store; Redis today, any conforming store tomorrow.

ISU drain seam

Stop new traffic, let calls finish, exit at zero — never migrate calls in flight.

From POC to product: the boundary moved, the seams got real

Dimension	POC · v1.2.0, Sept 2026	Product · v0.2.0
Carrier boundary	Local mocks stand in for the core	Operator S-SBC bridges a real SIP trunk
Transport	UDP only	UDP/TCP 5060 · TLS 5061 mTLS
SIP stack	in-process test stack	reSIProcate (C++) DUM production stack
Peer control	Source-IP allowlist only	mTLS + peer allowlist, fail-closed
State	In-process memory	Redis runtime + PostgreSQL governance
Engineering base	16 ADRs · 402 CI cases · ~65 open gaps	26+ ADRs · 1100+ tests across 4 gates
Delivery	Testbed, throwaway	Helm chart · air-gapped offline install

Decisions written down, behavior proven by tests

26+

Architectural Decision Records

Every non-trivial choice — protocol, state model, drain — recorded with context and consequences.

1100+

Automated test cases

Unit, integration, end-to-end and soak — all reproducible locally and in CI.

4

Quality gates in CI

One **make gate** runs lint, unit, integration, e2e and soak before merge.

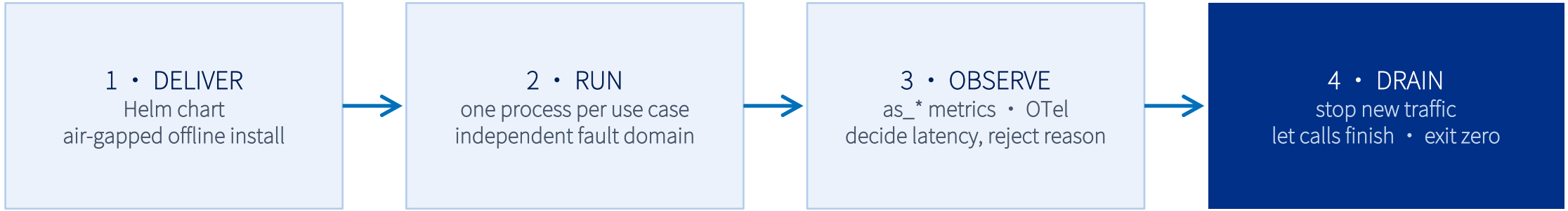
Docs follow code: ADR → config → contract test → release note. No specification drifts from implementation.
Built with AI-assisted R&D: code, tests and ADRs are AI-generated but human-reviewed — strict gates are the guardrail.

What ships with the product, not as post-sales docs

Compliance matrix	Which 3GPP and operator requirements apply — and exactly where the product explicitly stops.
NE datasheet	One-page product definition: protocols, ports, external dependencies and the resource envelope.
Fault demarcation	Core, S-SBC and AS — a clear line showing where each party's responsibility ends.
RACI	Who approves change orders, who operates, who holds keys — agreed before cutover.

These four are the contractual surface — they make integration and incident response unambiguous.

Delivered as Helm, operated like any network element



Fail-closed by design. If the peer allowlist cannot be read, the AS refuses new calls rather than open them.
No in-flight migration. Drain lets established calls complete; upgrade never rips a live call away mid-dialogue.

Carrier-grade operations: ISSU, redundancy, scale

ISSU upgrade	Rolling in-service upgrade — drain one replica, let its calls finish, upgrade, rejoin. No in-flight call migration (ADR-0009).
Redundancy	Intra-site N+1, no single point; cross-site 1+1 warm standby. Redis primary/replica + Sentinel (ADR-0008).
In-service growth / degrowth	Stateless multi-replica per use case, scaled in and out on Kubernetes; scale-down guarded until calls drain.
Microservice per use case	One process = one use case = one fault domain, one gray-release unit, one scale unit. State externalized to Redis.
UM / PM / FM telemetry	Prometheus text-format `/metrics` endpoint (as_* series) + OpenTelemetry; telemetry stays on-site.

Upgrade and scale happen without touching the IMS core — what a carrier-grade network element is expected to do.

One boundary, governed changes, data on your site

NETWORK BOUNDARY

The AS never enters the trust domain

- mTLS on TLS 5061 + peer allowlist, fail-closed
- No Diameter Sh, no CDR, no Rf interface
- Signalling only; SDP passes through untouched

CHANGE GOVERNANCE

Every rule change is an audited order

- Rules versioned in the control plane
- Each change order approved and logged
- Roll back to the previous version — no redeploy

DATA RESIDENCY

Data never leaves the operator's site

- Runtime and governance data stay on-prem
- Air-gapped offline install supported
- Held under PDPO and the operator's regime

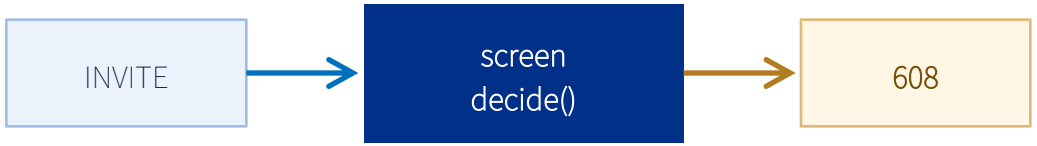
Two sample services prove the platform

SAMPLE 1 • NUMBER TRANSLATION B2BUA



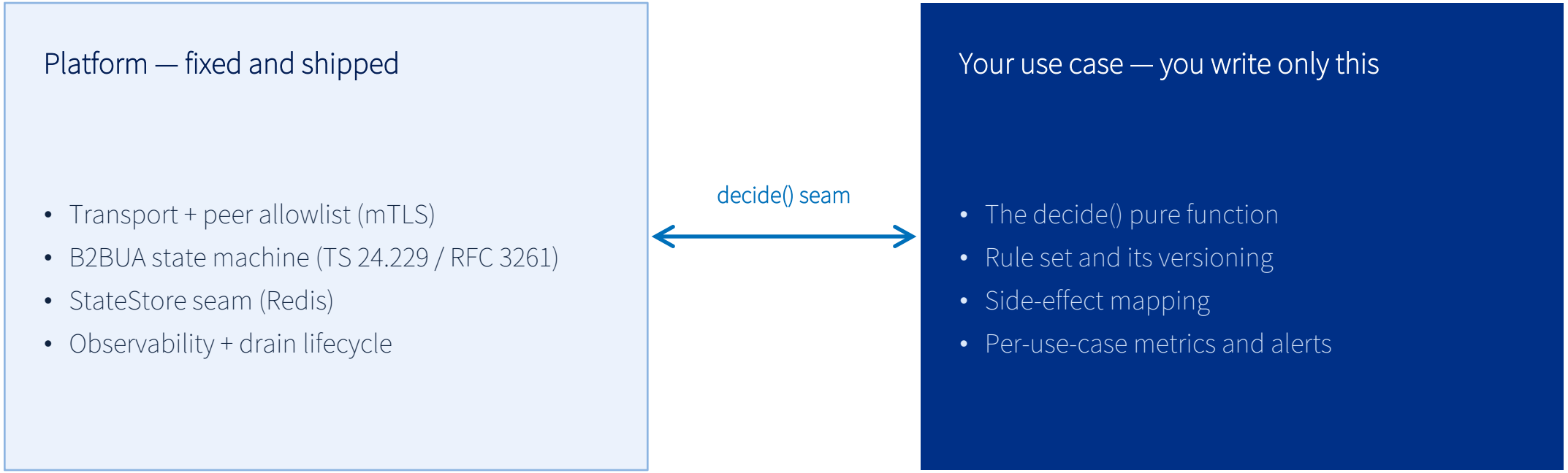
Rewrites the calling and called party between core and trunk as a full B2BUA. Two legs maintained independently; media and SDP pass through untouched.

SAMPLE 2 • ANTI-FRAUD SCREENING



On a suspected scam call, rejects with **608 Rejected** per **RFC 8688**. Established dialogs are left untouched; a flagged number self-clears after 5 minutes. Standard SIP — no special core feature.

Extensible by contract, not by forking the core



A new AS is a new decide() implementation + its rules — the platform is never forked, never patched.

WHERE WE ARE

v0.2.0 today, v1.1 next

TODAY · v0.2.0

- Two reference services on the platform
- Four CI gates green · 1100+ tests
- 26+ ADRs · docs in chain
- Helm, air-gapped install ready

NEXT · v1.1

- More reference AS use cases
- Richer control-plane workflows
- Carrier integration labs with S-SBC

Vendor-independent, on-premises, governed — and ready for the next service.